



Application Note

New Phase-Accurate Synchronization Of Two Vantage Systems Using the External Clock and Trigger Interfaces

Peter Kaczkowski
Verasonics Inc.

August 2015

Table of Contents

Table of Contents.....	2
Document Change History	2
1. Introduction	3
2. Procedure Summary	4
2.1 Setup the Hardware	4
2.2 Setup the Software	4
2.3 Verify Synchronous Operation and Align Phase	4
3. Theory of Operation	5
4. System Wiring Diagram	6
5. Script Programming for Sequence Synchronization	7
6. Example Script	9
6.1 Additional Scripting Notes.....	9
7. Verification and Phase Alignment.....	11
8. APPENDIX — Example script source code	12

Document Change History

Date	Section(s)	Description	Author
August 6, 2015	All	Rev A: Initial document	PK
August 10, 2015	Cover, 3, 4, 9	Rev B: After internal review — Added Document Number, Simplified procedure (3), clarified initialization (4), deleted some commented sections in the script that might be confusing (9).	PK

1. Introduction

Many ultrasound experiments require synchronization with external instruments such as lasers, motion stages in field mapping or robotically guided procedures, and synchronization with cardiac or respiratory phases. For these applications, the Verasonics system provides trigger input and output capability that is very straightforward to program in an acquisition sequence. Triggers provide timing synchronization accuracy on the order of 4 ns for output triggers, and four ADC sample clock periods for the input triggers. This accuracy is more than sufficient for many applications, but when combining two Vantage systems to combine channels for imaging with a large array, phase accurate synchronization between systems is required.

This document describes the hardware and software configuration needed to share a single 250 MHz system clock between two Vantage systems and thus enable synchronous and phase-aligned operation of the two hardware sequencers. This configuration permits coherent acquisitions with up to 512 channels (using two Vantage 256 systems; it is also possible to synchronize different Vantage models). Each Vantage unit operates with its host controller, and the data collected on each system can be processed locally for real time display. The data can also be transferred to a common computer during the sequence or after the experiment, and processed coherently using the complete data set.

This Application Note describes how the hardware must be connected to provide proper signal integrity, and how the software sequences must be arranged to assure proper operation. The L11-4v plane wave imaging example script “SetupL11_4vFlash.m” is used to illustrate how an acquisition sequence should be modified to enable synchronous operation, and the complete example script “SetupL11_4vFlash_2SysSync.m” is provided for configuration and testing. Finally, an oscilloscope-based test procedure is described to help the user verify that synchronization is working properly, and to precisely align the acquisitions in time, given the user’s specific Vantage hardware and interconnection cables.

Verasonics also offers an Multi-System Synchronization Module (“Sync module”) that permits synchronizing 2 to 8 chassis together, for up to 2048 coherent channels.

This external module greatly simplifies the hardware interconnection and the software implementation, and also significantly reduces clock skew between systems. The cable-only configuration, available for synchronization of no more than two systems, has the advantage of low hardware expense at the cost of some programming complexity and clock skew that must be adjusted.

2. Procedure Summary

2.1 Setup the Hardware

1. Set up the two Vantage systems and their computers next to each other. Be sure to leave a little space between systems for unimpeded air flow.
2. Connect HDMI cable and BNC cables as indicated in the wiring diagram, selecting one system as the MASTER.
3. Turn on both systems in any order, and launch Matlab on both computers.

2.2 Setup the Software

4. Install the provided `SetUpL11_4vFlash_2SysSync.m` example script on both computers.
5. On the MASTER, no change to the script is required.
6. Edit the SLAVE system script to properly identify the system as the SLAVE.
7. On the SLAVE, run the script without a transducer plugged in, and continue at the no transducer query so that the windows appear and the system enters the wait state.
8. Within 10 seconds, run the script on the MASTER (no transducer). If the SLAVE sequence DMA operation times out (waiting for data) and quits (script is set to wait up to 10 seconds), re-launch the SLAVE script so that it is ready before the MASTER.
9. Freeze or Quit the scripts using the MASTER controls. The SLAVE will not acquire additional frames without the MASTER's trigger, and thus the RcvData buffer will contain frames synchronous with those on the MASTER.

2.3 Verify Synchronous Operation and Align Phase

10. Connect two oscilloscope probes (one to each system) to transducer connector outputs on the same connector pins (active channels) and the ground clips to the shield spring. Launch the scripts as above.
11. Trigger the scope using the MASTER signal. It is helpful to set the high voltage slider on both systems above 10 V (11.3 V is obtained with a single click), and the trigger level to around 1 Volt.
12. Observe that the transmitter signals are not jittering or drifting with respect to each other. Measure the time delay between waveforms, if any. The SLAVE should be about 20 to 24 ns behind the MASTER, and the script is currently set for 20 ns, so only the residual difference should be detected.
13. Quit the MASTER script. Adjust the "master2slaveDelay" variable in the MASTER script as needed.
14. Restart the scripts, verifying that the phase alignment is within 4 ns (± 2 ns).

3. Theory of Operation

In a clock-synchronized configuration, the 250 MHz system clock is provided by one of the systems, designated the “Master”, via an HDMI cable plugged into the clock ports on the rear panels. When a Vantage hardware unit is initialized (e.g. by running the “Version” command from the Matlab command line), it uses the externally provided 250 MHz clock if it is present; otherwise, it will use its own internal system clock. Thus the systems can be powered up in any order, but the HDMI cable must be installed prior to initializing the Slave system (initialization occurs when running VSX, Version, etc.).

The two host controllers run independent software sequences, and these are event-aligned using the trigger interfaces: in every frame, both sequences enter a wait for trigger state, and then the Master system emits a trigger pulse to launch both systems at the same instant. After one frame of acquisitions, both systems enter the wait state again, and are re-triggered when the Master is ready. The frame rate is controlled by the Master sequence, and the Slave system simply has to have completed its frame acquisition, data transfer, and processing before the Master trigger occurs.

For imaging applications, for example with 512 channels connected to a 512-element array, the two acquisition sequences are nearly identical, with a small number of differences in sequence timing between Master and Slave code. For example, the Master may use Time To Next Acquisition (TTNA) Sequence controls to set the frame rate, allowing enough time to complete all Transmit/Receive and data transfer (DMA) hardware operations comprising an acquisition frame, and with some spare time to allow for responding to an occasional GUI control action. The Slave might accomplish all of the same actions, but should not use a TTNA constraint to allow the hardware sequence to progress as fast as it can. The Master sequence may also delay its acquisition to allow for the inevitable skew in clocks, given the propagation delays inherent in the HDMI cable. Completely different sequences are permissible, even those using different types of probes, though triggering alone may be a sufficient synchronization mechanism if phase-coherent processing is not a requirement.

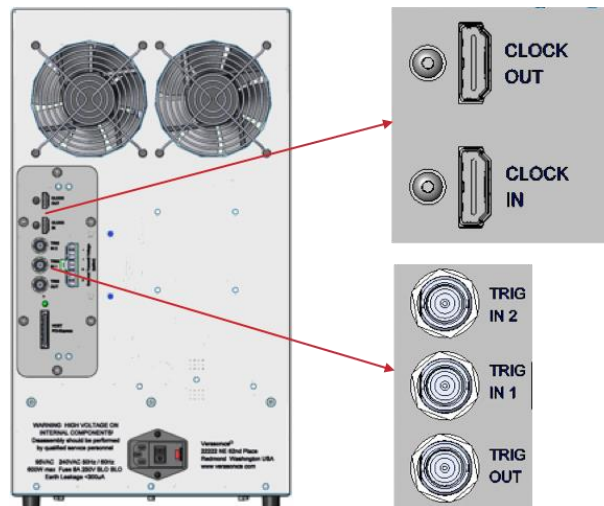
The fundamental requirement for sequence synchronization is that the Slave sequence must always have completed its acquisition frame by the time the Master is ready to acquire its next frame. This includes any external processing functions and HW/SW sync points. If it is not ready, the Slave will miss that frame and resynchronize with the following one. From that point on, the respective RcvData buffers will not longer contain time-aligned data. Multiple DMA transfers within a frame may also result in misalignment.

Combining data acquired by multiple systems can be done during sequence execution via the external function processing mechanism, or after sequence completion. Data may be transferred over a local network to one of the host computers, a third computer, or a network attached storage device. If data is to be transferred to nonvolatile storage or over a network within the sequence, it is important to consider both the typical and extreme times required for such transfers. If the transfer time is highly variable, an additional handshake mechanism may be required to ensure that the Master waits until the Slave is ready before triggering. Currently, hardware and software for gathering acquired data is not provided by Verasonics, and is the responsibility of the user.

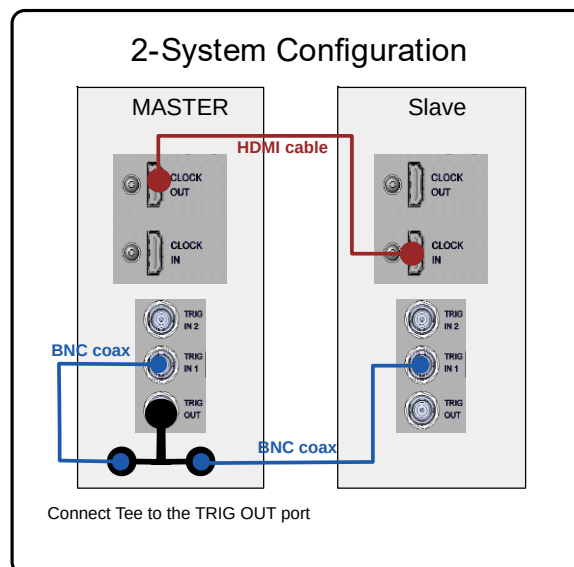
4. System Wiring Diagram

An external interface I/O panel on the back of the Vantage system unit contains Clock Input and Output ports using HDMI format connectors, and input and output trigger BNC connectors.

NOTE: Do not connect the CLOCK ports to a standard HDMI video device!!!



Hardware synchronization is achieved by connecting the master and slave systems as shown in the figure, and initializing the Slave after the Master is powered on, and the HDMI cable plugged in, to be sure that the clock is already present on the CLOCK IN port when the Slave sequencer initializes. Otherwise, the Slave will use its internal clock, and the system sequences may appear to run normally, but data frames will not be synchronized.



5. Script Programming for Sequence Synchronization

The Master system clock provides the 250 MHz clock signal for the slave system for phase accuracy. The individual systems must also align their sequences to the same event, and triggering and synchronization sequence control commands are used to accomplish this alignment. In the simplest configuration, both systems run the same (almost) sequence program, which contains several important modifications to a conventional script, as described below.

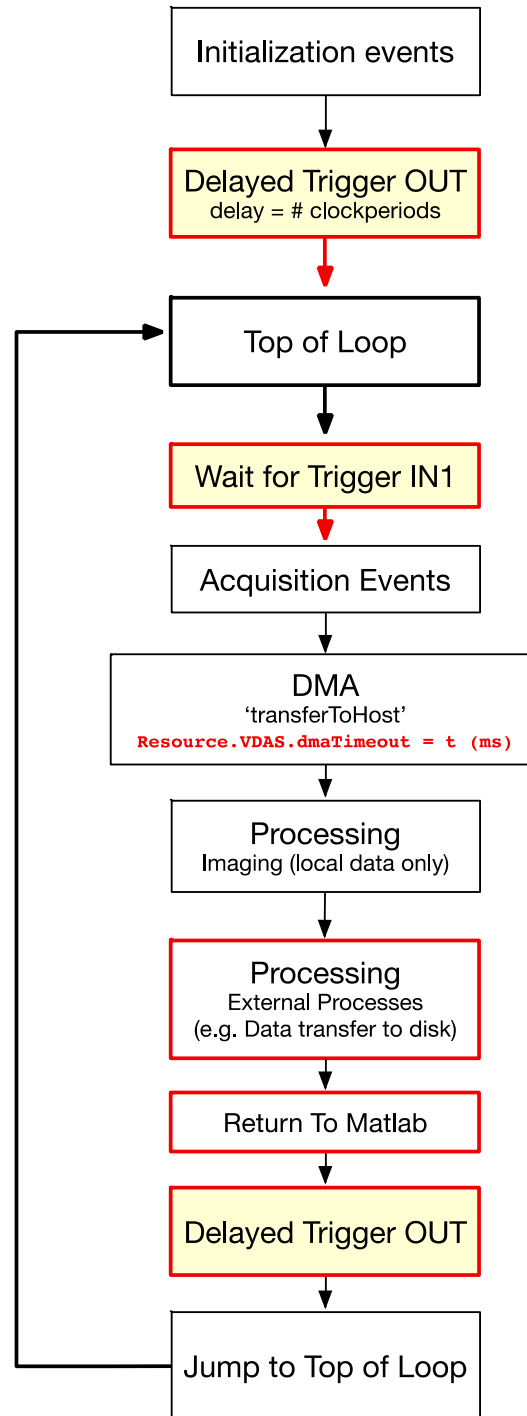
After initial setup events and upon entering the event loop, the synchronization sequence pauses the system, waiting for an external trigger input. The “start acquisition” trigger signal is provided by the Master trigger output: this delayed output trigger command is launched before entering the wait state, with a programmable delay that is long enough to permit the execution of the “pause for trigger in” event. Because both systems are paused waiting for the start trigger, and are sampling the trigger input using the same clock (though with phase delay), they both detect the trigger and begin acquisition within a fixed delay. To optimize phase coherence, use identical length coax cables for both systems.

If the two systems are executing different sequence types, it is important to note that both systems must receive at the same ADC frequency. In software versions 2.x, the ADC frequency is set by the choice of Trans.frequency and the Receive.samplesPerWave parameter. It is also essential that both systems be able to complete the frame before the start trigger for the next acquisition is issued.

Below is a flow chart describing the approach to software synchronization using a simple imaging script that loops over an acquisition, data transfer, and processing events. In RED are the new items required to convert a regular script to one that permits 2-system synchronization using an HDMI cable, 2 BNC coax cables, and a BNC Tee connector.

Major components of a sequence program are identified in the flowchart, but each “box” does not imply a separate event structure definition; indeed the “wait for trigger” should be programmed in the same event as the first acquisition event to minimize delay. The key synchronization logic uses the triggers, identified with yellow fill, and other modifications to a conventional script are identified in red.

Two-System Synchronization Program Flow



6. Example Script

The following script is adapted from the basic single-angle plane wave imaging script to minimize complexity. Pay particular attention to the sections delineated with the comment lines:

```
% === add for 2-system sync ===  
... New synchronization code here ...  
% ===
```

The program is designed to be installed on both the Master and Slave computers, but requires specifying which system is which using the variable “master”:

```
master = 0; % =1 Master  =0 Slave  Identify if this is the Master computer or the Slave
```

There are two differences between sequences, and both are related to timing. First, a fixed delay can be added to the Master transmit output to compensate for skew between the hardware trigger input sampling on both systems.

```
if master  
    master2slaveDelay = 0.020; % (microsecs) measured delay between the master and slave  
                                % Quantized in 4 ns increments.  
else  
    master2slaveDelay = 0;  
end
```

Second, the Master sets the frame rate using a Time to Next Acquisition Sequence Control (TTNA), while the Slaves should not: Slave sequencers should perform all operations as fast as they can in order to be “ready” before the Master completes its frame and issues the start trigger.

Key changes enabling Sync Module synchronization in the example script are documented in the code comments, and are listed here for convenience:

1. Set Master or Slave sequence using “master” variable
2. Set Master/Slave clock skew compensation time delay
3. Set “Resource.VDAS.dmaTimeout” time (currently 10 seconds)
4. Apply clock skew delay to TX structure
5. Define dummy transmit TX(2) structure for output trigger, with no acoustic output
6. Define triggerOut and triggerIn sequence controls
7. Add delayed triggerOut events using the dummy transmit TX(2), before the loop and at the bottom of the loop
8. Add “wait for triggerIn” and TTNA (Master only) to first acquisition event
9. Add ‘waitForTransferComplete’ sequence control to permit DMA timeout check

6.1 Additional Scripting Notes

- An explicit “ReturnToMatlab” sequence control is required when not jumping back to event number 1. An automatic “ReturnToMatlab” is only added by VSX when the jump goes to event #1.
- The trigger out delay argument is recognized in release 2.10 but was erroneously left undocumented. This mode is essential for assuring that a system can trigger itself.

- A ‘triggerIn’ sequence control will override (cancel) a TTNA timer in progress. There is no interaction with a TTNA in the same event because the TTNA applies to the next acquisition, after the trigger has been received. The TTNA timer starts at the beginning of the acquisition T/R, which only occurs after the pause (i.e., “wait for trigger in” is concluded.
- The Process structure in the provided example script sets the compression factor to 1 to be able to see input noise on the display, without a transducer connected. This facilitates observing whether the script is running. Normally, this parameter is set to 0.5 by default.

7. Verification and Phase Alignment

1. Connect two oscilloscope probes to transducer connector outputs on the same connector pins (active channels) and the ground clips to the shield spring.

It is often convenient to grasp a stiff inch-long wire conductor (e.g. a through-hole resistor leg) with the oscilloscope probe spring clip or “hat”, and hang it from the ZIF connector by inserting it into one pin socket. There will be no force other than slight friction holding it in place.

2. Trigger the scope using the MASTER signal. It is helpful to set the high voltage slider on both systems to 11.3 Volts, and the trigger level to around 1 Volt.

The value of 11.3 Volts is what one obtains by clicking in the slider field once.

3. Observe that the transmitter signals are not jittering with respect to each other. Measure the time delay between waveforms, if any. The SLAVE should be about 20 – 24 ns behind the MASTER, and the script is currently set for 20 ns, so only the residual difference should be detected.

The open circuit (no load) waveforms on the standard frequency system are “ugly”, but should look very similar to each other. The high frequency model has a different transmitter, characterized by a much faster (less inductive) transition time, and so HF system waveforms look more like textbook tristate rectangular waveforms. The key is to observe phase jitter (should be very little) and phase shift between systems.

4. Quit the MASTER script. Adjust the “master2slaveDelay” variable in the MASTER script as needed.

No adjustment to the Master script will likely be necessary.

5. Restart the scripts, verifying that the phase alignment is within 4 ns.

Note that using the provided script both systems put out a trigger pulse, but the Slave output is not connected. Indeed, the Vantage trigger output is always synchronized to the system’s *internal* system clock, so the Slave trigger will not be synchronized to the Master trigger, even when it is using the Master’s system clock for acquisition timing. For this reason, *one cannot verify that the systems are operating synchronously by comparing the two output trigger signals* using an oscilloscope. One must verify synchronous operation by examining the transmitter outputs directly.

8. APPENDIX — Example script source code:

SetUpL11_4vFlash_2SysSync_master.m

```
% SetUpL11_4vFlash_2SysSync.m - Example of phase synchronizing two systems by sharing the 250 MHz clock and triggers
%
% Based on SetUpL11_4vFlash imaging with single plane wave transmit.
% Description:
%   Sequence programming file for L11-4v Linear array, using a plane wave
%   transmit and single receive acquisition. All 128 transmit and receive
%   channels are active for each acquisition. Processing is asynchronous
%   with respect to acquisition.
%
% The original script was modified to use in a 2-system shared clock configuration, in which an HDMI cable
% passes the 250 MHz clock
% from the "master" system clock OUT to the "slave" system clock IN port, and BNC cables are used to
% trigger the sequences.
% The master system triggers the acquisition for both systems, using a delayed Trigger OUT command.
% See "Application Note Phase-Accurate Synchronization of Two Vantage Systems" for further documentation.
% Note the sections of code below identified by the comment: "% === add for 2-system sync ==="
%
% Testing: Tested with software release 2.10.4 on two Vantage 256 systems.
%
% Last update:
% 7/31/2015 - Add synchronization logic. Tested with Software 2.10.4.
% 9/3/2015 - Refined Master/Slave timing; Tested with 2.11.6
%
% Notice:
% This file is provided by Verasonics to end users as a programming
% example for the Verasonics Vantage Research Ultrasound System.
% Verasonics makes no claims as to the functionality or intended
% application of this program and the user assumes all responsibility
% for its use

clear all

filename = ('L11-4vFlash_2SysSync'); % assigning the matfile name to the variable 'filename' permits avoiding
the VSX query

% === add for 2-system sync ===
master = 1; % =1 Master =0 Slave Identify if this is the Master computer or the Slave
if master
    master2slaveDelay = 0.020; % (microsecs) measured delay between the master and slave start times.
                                % Must measure this delay for every installation because it will vary depending
                                % on cable lengths
                                % and other hardware details. Quantized in 4 ns increments.
else
    master2slaveDelay = 0;
end
Resource.VDAS.dmaTimeout = 10*1000; % (ms) time software sequencer will wait for 'transferToHost'
                                % set this time long enough to permit launching the Slave sequence
                                % and then launching the Master sequence
% ===

% Define system parameters.
Resource.Parameters.numTransmit = 128; % number of transmit channels.
Resource.Parameters.numRcvChannels = 128; % number of receive channels.
Resource.Parameters.speedOfSound = 1540; % set speed of sound in m/sec before calling computeTrans
Resource.Parameters.simulateMode = 0;
% Resource.Parameters.simulateMode = 1 forces simulate mode, even if hardware is present.
% Resource.Parameters.simulateMode = 2 stops sequence and processes RcvData continuously.

% Specify Trans structure array.
Trans.name = 'L11-4v';
Trans.units = 'mm'; % Explicit declaration avoids warning message when selected by default
Trans = computeTrans(Trans); % L11-4v transducer is 'known' transducer so we can use computeTrans.
Trans.maxHighVoltage = 50; % set maximum high voltage limit for pulser supply.

% Specify SFormat structure array.
SFormat.transducer = 'L11-4v'; % 128 element linear array
SFormat.scanFormat = 'RLIN'; % rectangular linear array scan
SFormat.radius = 0; % ROC for curved lin. or dist. to virt. apex
SFormat.theta = 0;
SFormat.numRays = 1; % no. of Rays (1 for Flat Focus)
SFormat.FirstRayLoc = [0,0,0]; % x,y,z
SFormat.rayDelta = 128*Trans.spacing; % spacing in radians(sector) or dist. between rays (wvlnghs)
SFormat.startDepth = 5; % Acquisition depth in wavelengths
```

```
SFormat.endDepth = 192; % This should preferably be a multiple of 128 samples.

% Specify PData structure array.
PData.sFormat = 1; % use first SFormat structure.
PData.pdeltaX = Trans.spacing;
PData.pdeltaZ = 0.5;
PData.Size(1) = ceil((SFormat.endDepth-SFormat.startDepth)/PData.pdeltaZ); % startDepth, endDepth and pdelta
set PData.Size.
PData.Size(2) = ceil((Trans.numelements*Trans.spacing)/PData.pdeltaX);
PData.Size(3) = 1; % single image page
PData.Origin = [-Trans.spacing*(Trans.numelements-1)/2,0,SFormat.startDepth]; % x,y,z of upper lft crnr.

% Specify Media object. 'pt1.m' script defines array of point targets.
pt1;
Media.function = 'movePoints';

% Specify Resources.
Resource.RcvBuffer(1).datatype = 'int16';
Resource.RcvBuffer(1).rowsPerFrame = 4096; % this size allows for maximum range
Resource.RcvBuffer(1).colsPerFrame = Resource.Parameters.numRcvChannels;
Resource.RcvBuffer(1).numFrames = 100; % 100 frames used for RF cineloop.
Resource.InterBuffer(1).datatype = 'complex';
Resource.InterBuffer(1).rowsPerFrame = 1024; % this is for greatest depth
Resource.InterBuffer(1).colsPerFrame = PData.Size(2);
Resource.InterBuffer(1).numFrames = 1; % one intermediate buffer needed.
Resource.ImageBuffer(1).datatype = 'double';
Resource.ImageBuffer(1).rowsPerFrame = 1024;
Resource.ImageBuffer(1).colsPerFrame = PData.Size(2);
Resource.ImageBuffer(1).numFrames = 10;
Resource.DisplayWindow(1).Title = filename;
Resource.DisplayWindow(1).pdelta = 0.35;
ScrnSize = get(0,'ScreenSize');
DwWidth = ceil(PData.Size(2)*PData.pdeltaX/Resource.DisplayWindow(1).pdelta);
DwHeight = ceil(PData.Size(1)*PData.pdeltaZ/Resource.DisplayWindow(1).pdelta);
Resource.DisplayWindow(1).Position = [250,(ScrnSize(4)-(DwHeight+150))/2, ... % lower left corner position
DwWidth, DwHeight];
Resource.DisplayWindow(1).ReferencePt = [PData.Origin(1),PData.Origin(3)]; % 2D imaging is in the X,Z plane
Resource.DisplayWindow(1).Colormap = gray(256);

% Specify Transmit waveform structure.
TW.type = 'parametric';
TW.Parameters = [Trans.frequency, .67, 2, 1]; % A, B, C, D

% Specify TX structure array.
TX(1).waveform = 1; % use 1st TW structure.
TX(1).Origin = [0,0,0,0,0]; % flash transmit origin at (0,0,0).
TX(1).focus = 0;
TX(1).Steer = [0,0,0,0]; % theta, alpha = 0.
TX(1).Apod = ones(1,Trans.numelements);
TX(1).Delay = computeTXDelays(TX(1));
% === add for 2-system sync ===
extraDelay = master2slaveDelay * Trans.frequency;
TX(1).Delay = TX(1).Delay + extraDelay; % extraDelay must be zero for the slave system

TX(2) = TX(1); % dummy transmit struture that does not turn on transmitters but does allow
a trigger out
TX(2).Apod = zeros(1,Trans.numelements);
% ===

% Specify TGC Waveform structure.
TGC.CntrlPts = [0,200,344,452,606,747,870,920];
TGC.rangeMax = SFormat.endDepth;
TGC.Waveform = computeTGCWaveform(TGC);

% Specify Receive structure arrays -
% endDepth - add additional acquisition depth to account for some channels
% having longer path lengths.
% InputFilter - The same coefficients are used for all channels. The
% coefficients below give a broad bandwidth bandpass filter.
maxAcqLength = sqrt(SFormat.endDepth^2 + (Trans.numelements*Trans.spacing)^2) - SFormat.startDepth;
wlsPer128 = 128/(4*2); % wavelengths in 128 samples for 4 samplesPerWave
Receive = repmat(struct('Apod', ones(1,Trans.numelements), ...
'startDepth', SFormat.startDepth, ...
'endDepth', SFormat.startDepth + wlsPer128*ceil(maxAcqLength/wlsPer128), ...
'TGC', 1, ...
'bufnum', 1, ...
'framenum', 1, ...
'acqNum', 1, ...
'samplesPerWave', 4, ...
```

```

        'mode', 0, ...
        'callMediaFunc', 1), 1, Resource.RcvBuffer(1).numFrames);
% - Set event specific Receive attributes.
for i = 1:Resource.RcvBuffer(1).numFrames
    % -- Acquisition for full frame.
    Receive(i).framenum = i;
end

% Specify Recon structure arrays.
Recon = struct('sencutoff', 0.6, ...
    'pdatanum', 1, ...
    'rcvBufFrame', -1, ... % use most recently transferred frame
    'IntBufDest', [1,1], ...
    'ImgBufDest', [1,-1], ... % auto-increment ImageBuffer each recon
    'RINums', 1);

% Define ReconInfo structures.
ReconInfo = repmat(struct('mode', 0, ...
    'txnum', 1, ...
    'rcvnum', 1, ...
    'regionnum', 0), 1, 1);

% Specify Process structure array.
pers = 20;
Process(1).classname = 'Image';
Process(1).method = 'imageDisplay';
Process(1).Parameters = {'imgbufnum', 1, ... % number of buffer to process.
    'framenum', -1, ... % (-1 => lastFrame)
    'pdatanum', 1, ... % number of PData structure to use
    'norm', 1, ... % normalization method (1 means fixed)
    'pgain', 3.0, ... % pgain is image processing gain
    'persistMethod', 'simple', ...
    'persistLevel', pers, ...
    'interp', 1, ... % method of interpolation (1=4pt interp)
    'compression', 1, ... % == set to 1 for use without transducer (normally 0.5)
    'reject', 5, ... % reject level
    'mappingMode', 'full', ...
    'display', 1, ... % display image after processing
    'displayWindow', 1};

% Specify SeqControl structure arrays.
SeqControl(1).command = 'jump'; % jump back to event in argument set below
JUMP = 1;
SeqControl(2).command = 'timeToNextAcq'; % time between frames
SeqControl(2).argument = 20*1000; % 20 msec for 50 fps
TTNA = 2;
SeqControl(3).command = 'returnToMatlab';
RTNMatlab = 3;
nsc = 4; % nsc-1 is count of SeqControl objects

% === add for 2-system sync ===
SeqControl(nsc).command = 'triggerOut';
SeqControl(nsc).condition = 'syncSYNC_CLK';
SeqControl(nsc).argument = 10*250; % 250 counts per microsecond (up to about 4100 microsecs)
TrigOUT = nsc;
nsc = nsc+1;

SeqControl(nsc).command = 'triggerIn';
SeqControl(nsc).condition = 'Trigger_1_Rising';
SeqControl(nsc).argument = 0; % (integer: 250 ns counts ... 8-bit counter) =0 waits forever
TrigIN1 = nsc;
nsc = nsc+1;
% ===

% SeqControl(nsc).command = 'sync'; % synchronize SW and HW sequencers (optional)
% SYNC = nsc;
% nsc = nsc+1;

n = 1; % n is count of Events

% === add for 2-system sync ===
Event(n).info = 'Initial delayed Trigger OUT';
Event(n).tx = 2; % transmit or receive required to get a trigger out; set TX.Apod = 0 to turn off
transmitters
Event(n).rcv = 0; % first Rcv to be overwritten later
Event(n).recon = 0; % no Recon
Event(n).process = 0;
Event(n).seqControl = TrigOUT; % trigger command
n=n+1;

```

```
SeqControl(1).argument = n; % set jump back to top of loop
% ===

% Acquire all frames defined in RcvBuffer
for i = 1:Resource.RcvBuffer(1).numFrames

    Event(n).info = 'Full aperture acquisition';
    Event(n).tx = 1; % use 1st TX structure.
    Event(n).rcv = i; % use ith Rcv structure for the ith frame.
    Event(n).recon = 0; % no reconstruction.
    Event(n).process = 0; % no processing
    % === add for 2-system sync ===
    if master
        Event(n).seqControl = [TrigIN1, TTNA]; % === placing the "wait for trigger" in the same event as
        T/R minimizes delay to about 0.7 microseconds % === triggerIN cancels any running TTNA timer (but allows
        setting TTNA for next transmit)
    else
        Event(n).seqControl = [TrigIN1]; % === No TTNA for the Slave system, so that it runs faster than
        the Master sequence
    end
    % ===
    n = n+1;

    Event(n).info = 'DMA';
    Event(n).tx = 0; % use 1st TX structure.
    Event(n).rcv = 0; % use ith Rcv structure for the ith frame.
    Event(n).recon = 0; % no reconstruction.
    Event(n).process = 0; % no processing
    Event(n).seqControl = [nsc,nsc+1]; %seqControl struct defined below.
    SeqControl(nsc).command = 'transferToHost';
    % === add for 2-system sync ===
    SeqControl(nsc+1).command = 'waitForTransferComplete'; % used to inform the user that acquisition
    hasn't occurred, % presumably because a trigger was not
    received within DMA timeout
    SeqControl(nsc+1).argument = nsc;
    nsc = nsc + 2;
    % ===
    n = n+1;

    Event(n).info = 'Reconstruct & Process';
    Event(n).tx = 0; % no transmit
    Event(n).rcv = 0; % no rcv
    Event(n).recon = 1; % reconstruction
    Event(n).process = 1; % processing
    if floor(i/5) == i/5 % Exit to Matlab every 5th frame
        Event(n).seqControl = RTNMatlab; % return to Matlab
    else
        Event(n).seqControl = 0;
    end
    n = n+1;

    % === add for 2-system sync ===
    Event(n).info = 'delayed Trigger OUT';
    Event(n).tx = 2; % dummy TX
    Event(n).rcv = 0;
    Event(n).recon = 0;
    Event(n).process = 0;
    Event(n).seqControl = TrigOUT; % delayed trigger out (no TTNA because it is cancelled by wait for trigger
    in)
    % Event(n).seqControl = [SYNC, TrigOUT]; % trigger command always executes after sync
    % % if SYNC (optional) times out, DMA + Recon + Process
    must be taking longer than expected
    % % SYNC should only be used on the Master, and only if the
    frame rate is % to be controlled by software processes (e.g. external
    % functions)
    n = n+1;
    % ===
end

Event(n).info = 'Jump back';
Event(n).tx = 0; % no TX
Event(n).rcv = 0; % no Rcv
Event(n).recon = 0; % no Recon
Event(n).process = 0;
```

```

Event(n).seqControl = JUMP; % jump command (must have explicit 'returnToMatlab' in sequence if not to event
#1)

% User specified UI Control Elements
% - Sensitivity Cutoff
UI(1).Control = {'UserB7','Style','VsSlider','Label','Sens. Cutoff',...
                'SliderMinMaxVal',[0,1.0,Recon(1).senscutoff],...
                'SliderStep',[0.025,0.1],'ValueFormat','%1.3f'};
UI(1).Callback = text2cell('%-UI#1Callback');

% - Range Change
UI(2).Control = {'UserA1','Style','VsSlider','Label','Range',...
                'SliderMinMaxVal',[64,320,SFormat.endDepth],'SliderStep',[0.1,0.2],'ValueFormat','%3.0f'};
UI(2).Callback = text2cell('%-UI#2Callback');

% Specify factor for converting sequenceRate to frameRate.
frameRateFactor = 5;

% Save all the structures to a .mat file.
save(filename);
VSX % setting 'filename' variable permits running automatically upon running the setup script
return

% **** Callback routines to be converted by text2cell function. ****
%-UI#1Callback - Sensitivity cutoff change
ReconL = evalin('base','Recon');
for i = 1:size(ReconL,2)
    ReconL(i).senscutoff = UIValue;
end
assignin('base','Recon',ReconL);
Control = evalin('base','Control');
Control.Command = 'update&Run';
Control.Parameters = {'Recon'};
assignin('base','Control',Control);
return
%-UI#1Callback

%-UI#2Callback - Range change
simMode = evalin('base','Resource.Parameters.simulateMode');
% No range change if in simulate mode 2.
if simMode == 2
    set(hObject,'Value',evalin('base','SFormat.endDepth'));
    return
end
range = UIValue;
assignin('base','range',range);
SFormat = evalin('base','SFormat');
SFormat.endDepth = range;
assignin('base','SFormat',SFormat);
evalin('base','PData.Size(1) = ceil((SFormat.endDepth-SFormat.startDepth)/PData.pdeltaZ);');
evalin('base','[PData.Region,PData.numRegions] = createRegions(PData);');
evalin('base','Resource.DisplayWindow(1).Position(4)
ceil(PData.Size(1)*PData.pdeltaZ/Resource.DisplayWindow(1).pdeltaZ);');
Receive = evalin('base','Receive');
Trans = evalin('base','Trans');
maxAcqLength = sqrt(range^2 + (Trans.numElements*Trans.spacing)^2)-SFormat.startDepth;
wlsPer128 = 128/(4*2);
for i = 1:size(Receive,2)
    Receive(i).endDepth = SFormat.startDepth + wlsPer128*ceil(maxAcqLength/wlsPer128);
end
assignin('base','Receive',Receive);
evalin('base','TGC.rangeMax = SFormat.endDepth;');
evalin('base','TGC.Waveform = computeTGCWaveform(TGC);');
evalin('base','if VDAS==1, Result = loadTgcWaveform(1); end');
Control = evalin('base','Control');
Control.Command = 'update&Run';
Control.Parameters = {'SFormat','PData','Receive','Recon','DisplayWindow','ImageBuffer'};
assignin('base','Control',Control);
assignin('base','action','displayChange');
return
  
```


*This document is intended for use by Verasonics customers only.
Do not duplicate or distribute without permission.*

Verasonics, Inc.
11335 NE 122nd way, Suite 100
Kirkland, WA 98034
USA